Key Design Decisions

A. How environmental spawn effects are dispatched when creatures enter the map

Design 1: Each spawner (Hole, Vent, tree, infection status) directly applies the spawn effect inline after placing the creature. For example, Hole contains code to increase Undead max HP, drop worker items for Slime, etc.

Pros	Cons
Simple and direct; no additional classes needed.	Every spawner must duplicate the spawn-effect logic for each creature type, violating the DRY principle.
	Adding a new creature type requires modifying every spawner class, violating the Open-Closed Principle.
	Spawners become tightly coupled to creature-specific behaviour, violating the Single Responsibility Principle.
	Other REQs (trees, infection, Alien Cubes) that spawn creatures must also duplicate the same effect logic.

Design 2: Use a SpawnEffectManager singleton with a registry pattern. Each creature type registers a SpawnEffect (functional interface) at initialisation. After any spawner successfully places a creature, it calls SpawnEffectManager.onCreatureSpawned(), which looks up and applies the registered effect by creature class.

Pros	Cons
Spawn effects are defined once per creature type and applied consistently regardless of the source (Hole, Vent, tree, infection), satisfying the DRY principle.	Requires an additional manager class and interface, adding structural complexity.
New creature types can register their own effects without modifying any existing spawner or effect class (Open-Closed Principle).	The registry lookup uses Class as a key, which does not support subclass matching (exact class match only).
Spawners only know how to create and place creatures; they have no knowledge of what environmental reactions occur (Single Responsibility Principle).	The indirection through a singleton may make the control flow less immediately visible.

The SpawnEffect functional interface provides a single integration point for all systems across multiple REQs (REQ3 trees, REQ2 Alien Cubes, REQ4 spawners and infection) to call.	

B. How the Parasite discovers infectable targets polymorphically

Design 1: InfectBehaviour uses instanceof checks to test each adjacent actor and item against known infectable types (ContractedWorker, Undead, Lantern, Cookies) and applies the appropriate infection directly.

Pros	Cons
No additional interface needed; uses existing concrete types.	Tightly couples InfectBehaviour to every infectable class, violating the Dependency Inversion Principle.
	Adding a new infectable target (e.g., a new item from another REQ) requires modifying InfectBehaviour, violating the Open-Closed Principle.
	The use of instanceof is considered an anti-pattern in OOP as it bypasses polymorphic dispatch.

Design 2: Create an Infectable interface with an onInfected(Actor, GameMap) method. Each target implements it and defines its own infection response. InfectBehaviour discovers targets using the engine's getActorAs(Infectable.class) and getItemsAs(Infectable.class), which leverage GameEntity.asCapability() for polymorphic lookup.

Pros	Cons
InfectBehaviour depends only on the Infectable abstraction, not on any concrete class (Dependency Inversion Principle).	Each infectable class must implement the interface, adding a method to multiple existing classes.
New infectable targets can be added by implementing Infectable and registering nothing else — the Parasite will automatically discover them (Open-Closed Principle).	The Infectable interface works through asCapability(), which requires it to be an interface or abstract class.

Each target defines its own infection response in onInfected(), following the Single Responsibility Principle.	
Leverages the engine's existing asCapability() system (getActorAs, getItemsAs) for zero-cost discovery, reusing established infrastructure rather than inventing ad-hoc mechanisms.	

C. How different infection effects are applied to different host types

Design 1: A single InfectedStatus class that stores the host type and uses conditional logic (if/else or switch) to apply the correct effect each tick (damage workers, drain lantern oil, reduce cookies, etc.).

Pros	Cons
Only one status class to maintain.	The class grows with every new host type, accumulating unrelated logic and violating the Single Responsibility Principle.
	Adding a new host type requires modifying the existing status class, violating the Open-Closed Principle.
	Requires instanceof or type-checking inside tickStatus(), which is an OOP anti-pattern.

Design 2: Separate status classes per host category (WorkerInfectedStatus, LanternInfectedStatus, CookieInfectedStatus). Each host's onInfected() method creates and attaches the appropriate status. The engine's tickStatuses() mechanism ticks them automatically.

Pros	Cons
Each status class handles exactly one infection effect (Single Responsibility Principle).	More classes to create and maintain (three instead of one).
New host types can add their own status without modifying existing ones (Open-Closed Principle).	Structural similarity between the status classes (e.g., the Parasite-spawning helper method is duplicated in WorkerInfectedStatus and CookieInfectedStatus).

No instanceof or type-checking needed; the host polymorphically selects the correct status in its onInfected() method.	
Each status leverages the engine’s existing Status interface and tickStatuses() lifecycle, integrating cleanly with the framework.	